

Permisos App

h_app · Documentación técnica completa

React Native · Expo SDK 54 · Firebase · TypeScript · RBAC

Jorge Alejandro Martinez Vazquez

Desarrollo de Aplicaciones Móviles Avanzada

Documentación completa — Permisos App (h_app)

React Native · Expo SDK 54 · Firebase · TypeScript · RBAC

Autor: Jorge Alejandro Martinez Vazquez

Índice

- Visión general
- Setup y README
- Arquitectura del sistema
- Firebase — Documentación técnica
- Variables de entorno
- Sistema RBAC
- Componentes, Hooks y Tipos
- Pantallas

1. Visión general

App móvil React Native / Expo con sistema de roles y permisos (RBAC) en tiempo real sobre Firebase.

¿Qué hace esta app?

Permisos App es una aplicación móvil que permite controlar **quién puede ver qué** dentro de la app, sin necesidad de publicar una nueva versión. Todo se administra en tiempo real desde un panel de control:

- Un administrador puede **bloquear un usuario** y ese usuario es desconectado en segundos
- Puedes **activar o desactivar pantallas** completas para grupos de usuarios
- Puedes **renombrar pantallas** y el cambio se refleja al instante para todos
- Los **permisos se definen una vez** y se reutilizan en cualquier pantalla o componente

Ejemplo práctico:

```
Administrador entra al panel → asigna el rol "Gerente" a un usuario
→ ese usuario ahora puede ver la pantalla de Reportes
→ sin reiniciar la app, sin nueva versión, en tiempo real
```

Stack tecnológico

Tecnología	Versión	Para qué se usa
Expo	SDK 54	Framework base — builds, assets, plugins nativos

Tecnología	Versión	Para qué se usa
React Native	0.81	UI multiplataforma (iOS, Android, Web)
Expo Router	v6	Navegación basada en archivos (file-based routing)
TypeScript	5.9	Tipado estático en todo el proyecto
Firebase Auth	v12	Autenticación de usuarios
Firestore	v12	Base de datos en tiempo real — roles, permisos, logs
Firebase Admin SDK	v13	Script de inicialización (solo servidor/Node.js)
React	19	Librería de UI

Arquitectura en 30 segundos



El modelo RBAC funciona así:

```

Permission → key única (ej: "manage:users")
  ↑
Role → array de permissionIds
  ↑
AppUser → roleIds[] + permissionIds[] directos
        Permisos finales = union de ambos

```

Colecciones en Firestore

Colección	Descripción
users	Usuarios con sus roles y permisos asignados

Colección	Descripción
<code>roles</code>	Roles disponibles (Administrador, Gerente, etc.)
<code>permissions</code>	Permisos atómicos con una key única
<code>screens</code>	Configuración de pantallas — activas/inactivas, permisos requeridos
<code>logs</code>	Auditoría inmutable de todas las acciones
<code>sessions</code>	Registro de sesiones activas e históricas
<code>app_config</code>	Configuración global de la app

Roles creados por defecto

Rol	Acceso
System Master	Acceso total — gestiona colecciones base
Administrador	Gestión completa de usuarios y pantallas
Gerente	Lectura de usuarios, roles y logs
Visualizador	Solo lectura básica

Pantallas principales

Pantalla	Ruta	Permiso requerido
Home / Tabs	<code>(tabs)/</code>	Solo estar autenticado
Gestión de usuarios	<code>pantallas/usuarios</code>	<code>manage:users</code>
Gestión de roles	<code>pantallas/roles</code>	<code>manage:roles</code>
Gestión de pantallas	<code>pantallas/pantallas</code>	<code>manage:screens</code>
Logs de auditoría	<code>pantallas/logs</code>	<code>read:logs</code>
Panel System Master	<code>pantallas/system-master</code>	<code>system:master</code>

Seguridad

Las reglas de Firestore (`firestore.rules`) son la última línea de defensa. Ningún cliente puede saltarlas aunque modifique el código de la app.

Reglas clave:

- Los usuarios solo pueden editar su propio documento, pero **nunca** pueden cambiar sus propios roles, permisos, o estado de bloqueo
- Los logs son **inmutables** — no se pueden editar ni borrar desde el cliente
- Solo `system:master` puede crear nuevos permisos o modificar `app_config`

Archivos clave del proyecto

Archivo	Propósito
lib/firebase.ts	Inicialización de Firebase con variables de entorno
lib/types.ts	Todos los tipos TypeScript del proyecto
context/AuthContext.tsx	Estado global de autenticación + resolución de permisos en tiempo real
hooks/useAuthHooks.ts	usePermission , useScreenGuard , useAuthRedirect
seed.js	Script de inicialización de Firestore (se corre una sola vez)
firestore.rules	Reglas de seguridad de Firestore

Setup inicial (resumen)

```
# 1. Instalar dependencias
npm install

# 2. Crear .env con credenciales de Firebase
# (ver sección Variables de entorno)

# 3. Inicializar datos en Firestore (una sola vez)
GOOGLE_APPLICATION_CREDENTIALS=./service-account.json node seed.js

# 4. Correr la app
npx expo start
```

Recursos y referencias

Documentación oficial

- **Expo Docs** — documentación completa del SDK, plugins, EAS Build
- **Expo Router** — enrutamiento basado en archivos
- **React Native Docs**
- **Firebase Docs — Firestore**
- **Firebase Docs — Auth**
- **Firebase Docs — Security Rules**
- **TypeScript Handbook**

Videos tutoriales

- **Firebase React Native Expo Authentication Setup (2024)**
- **React Native Firebase Authentication with Expo Router — Galaxies.dev (2024)**
- **File Based Routing | Expo Router Tutorial (2025)**
- **Complete Expo Router Bottom Tabs Tutorial (2025)**
- **React Native Tutorial: Firebase Firestore CRUD (2024)**

2. Setup y README

Requisitos

- Node.js 18+
- Expo CLI (`npm install -g expo-cli`)
- Cuenta Firebase con proyecto activo
- Archivo `.env` con variables de Firebase

Scripts disponibles

```
npm run start          # Inicia el servidor de desarrollo
npm run android        # Abre en emulador Android
npm run ios            # Abre en simulador iOS
npm run web            # Abre en navegador
npm run lint           # Corre ESLint
npm run reset-project  # Limpia el proyecto
```

Estructura de carpetas

```

h_app/
├── app/                                # Pantallas (file-based routing con Expo Router)
│   ├── _layout.tsx                    # Layout raíz – envuelve con AuthProvider
│   ├── (tabs)/
│   │   ├── _layout.tsx                # Tabs principales (nombres desde Firestore)
│   │   ├── bienvenida/                # Pantallas públicas (sin auth)
│   │   ├── login/                     # Pantallas de login
│   │   └── pantallas/                 # Panel de administración RBAC
│   │       ├── usuarios.tsx
│   │       ├── roles.tsx
│   │       ├── pantallas.tsx
│   │       ├── logs.tsx
│   │       └── system-master.tsx
│   └── lib/
│       ├── firebase.ts                # Inicialización y exports de Firebase
│       ├── types.ts                   # Todos los tipos TypeScript del proyecto
│       ├── context/
│       │   ├── AuthContext.tsx        # Proveedor de autenticación + permisos resueltos
│       ├── hooks/
│       │   ├── useAuthHooks.ts        # usePermission, useScreenGuard, useAuthRedirect
│       ├── services/
│       │   ├── authService.ts
│       │   ├── userService.ts
│       │   ├── screenService.ts
│       │   └── logService.ts
│       ├── assets/
│       ├── docs/
│       ├── seed.js
│       ├── firestore.rules
│       ├── app.json
│       ├── package.json
│       └── tsconfig.json

```

Asignar primer administrador

Después de correr el seed, asigna el rol de administrador a tu usuario desde Firebase Console:

- Ir a **Firestore** → **colección users** → **tu UID**
- Editar el documento y agregar:

```

{
  "roleIds": ["role_admin"],
  "isActive": true,
  "isBlocked": false
}

```

Para acceso total (System Master):

```

{
  "roleIds": ["role_system_master"]
}

```

■ ■ **Nunca subas** `service-account.json` **a git** — agrégalos a `.gitignore`

■ ■ *Nunca subas* `.env` *con tus credenciales Firebase*

3. Arquitectura del sistema

Visión general

Permisos App es una aplicación Expo/React Native con autenticación y control de acceso por roles (RBAC) gestionado enteramente desde Firebase. No tiene backend propio — toda la lógica de negocio corre en el cliente con Firestore como fuente de verdad.

Diagrama de arquitectura

```
graph TD
    subgraph App["App (React Native / Expo)"]
        Layout["app/_layout.tsx\nLayout raíz"]
        AuthProv["AuthProvider\ncontext/AuthContext.tsx"]
        Tabs["(tabs)/ Layout\nNombres desde Firestore"]
        Pantallas["pantallas/\nPanel Admin RBAC"]
        Hooks["hooks/useAuthHooks.ts\nusePermission · useScreenGuard · useAuthRedirect"]
    end

    subgraph Firebase
        FAuth["Firebase Auth\nEmail/Password"]
        Firestore["Firestore DB"]
        subgraph Colecciones
            Users["users/"]
            Roles["roles/"]
            Perms["permissions/"]
            Screens["screens/"]
            Logs["logs/"]
            Sessions["sessions/"]
            AppConf["app_config/"]
        end
    end

    Layout --> AuthProv
    AuthProv --> FAuth
    AuthProv --> Users
    AuthProv --> Roles
    AuthProv --> Perms
    Tabs --> Screens
    Pantallas --> Hooks
    Hooks --> AuthProv
    Pantallas --> Users
    Pantallas --> Roles
    Pantallas --> Perms
    Pantallas --> Screens
    Pantallas --> Logs
```

Diagrama de flujo de autenticación


```
sequenceDiagram
    participant U as Usuario
    participant App
    participant Auth as Firebase Auth
    participant FS as Firestore

    U->>App: Abre la app
    App->>Auth: onAuthStateChanged()
    Auth-->>App: firebaseUser (o null)

    alt No autenticado
        App->>U: Redirige a /bienvenida
    else Autenticado
        App->>FS: onSnapshot(users/{uid})
        FS-->>App: AppUser document

        alt isBlocked o !isActive
            App->>Auth: signOut()
            App->>U: Redirige a /bienvenida
        else Usuario válido
            App->>FS: getDocs(roles donde id in roleIds)
            FS-->>App: Roles del usuario
            App->>FS: getDocs(permissions donde id in permIds)
            FS-->>App: Permisos resueltos
            App->>App: setState({ resolvedPermissionKeys: Set })
            App->>U: Redirige a /(tabs)
        end
    end
end
```

Diagrama del modelo de datos RBAC

```

erDiagram
    AppUser {
        string uid PK
        string email
        string displayName
        boolean isActive
        boolean isBlocked
        string[] roleIds FK
        string[] permissionIds FK
    }

    Role {
        string id PK
        string name
        boolean isActive
        string[] permissionIds FK
        number order
    }

    Permission {
        string id PK
        string key
        string name
        string category
        boolean isActive
    }

    Screen {
        string id PK
        string routePath
        string displayName
        boolean isActive
        string[] requiredPermissions FK
    }

    Log {
        string id PK
        string userId FK
        string action
        string targetCollection
        Timestamp createdAt
    }

    AppUser }o--o{ Role : "roleIds"
    AppUser }o--o{ Permission : "permissionIds directos"
    Role }o--o{ Permission : "permissionIds"
    Screen }o--o{ Permission : "requiredPermissions"
    AppUser ||--o{ Log : "userId"

```

Estructura de navegación (Expo Router)

```

app/
  _layout.tsx      ← Stack raíz + AuthProvider + useAuthRedirect
  bienvenida/
    bienvenida.tsx ← Pantalla de bienvenida (pública)
  login/
    login.tsx      ← Formulario de login (pública)
  (tabs)/
    _layout.tsx    ← Bottom tabs – nombres y orden desde Firestore
    index.tsx      ← Tab principal (home)
  pantallas/
    usuarios.tsx   ← manage:users
    roles.tsx      ← manage:roles
    pantallas.tsx  ← manage:screens
    logs.tsx       ← read:logs
    system-master.tsx ← system:master

```

Decisiones técnicas

Por qué Firestore con `onSnapshot` en vez de REST polling

Los permisos y el estado de bloqueo de usuarios cambian en tiempo real desde el panel de administración. Con `onSnapshot`, el cliente recibe los cambios inmediatamente sin necesidad de reiniciar sesión ni hacer polling. Si un admin bloquea a un usuario, ese usuario es desconectado en segundos.

Por qué los permisos se resuelven en memoria (no se persisten en Firestore)

Persistir `resolvedPermissionKeys` en el documento del usuario crearía un problema de consistencia: si se agrega un permiso a un rol, habría que actualizar todos los usuarios con ese rol. En cambio, resolverlos en memoria al cargar la sesión garantiza que siempre estén actualizados con el estado real de roles y permisos.

Por qué TypeScript strict mode

El proyecto usa `"strict": true` en `tsconfig.json`. Esto evita errores comunes con `null` / `undefined` en los datos de Firestore, que pueden ser `null` si los documentos no existen o tienen campos opcionales.

Por qué `chunkArray` en lugar de una sola query

Firestore limita las consultas `where('__name__', 'in', [...])` a **10 elementos** por query. La función `chunkArray` divide los arrays de IDs en grupos de 10 y hace múltiples queries en paralelo.

Sobre `service-account.json`

El archivo `service-account.json` en el repo es para uso local del script `seed.js` durante el setup inicial. **Debe estar en `.gitignore`** y nunca subirse a ningún repositorio o entorno de CI/CD.

4. Firebase — Documentación técnica

Proyecto Firebase: **des-app-movil**

Servicios utilizados

Servicio	Uso
Firebase Auth	Autenticación de usuarios (email/password)
Firestore	Base de datos principal — usuarios, roles, permisos, logs
Firebase Admin SDK	Script de seed (<code>seed.js</code>) — solo en servidor

Inicialización (`lib/firebase.ts`)

```
import { getApps, initializeApp } from "firebase/app";
import { getAuth } from "firebase/auth";
import { getFirestore } from "firebase/firestore";

const app = getApps().length === 0 ? initializeApp(firebaseConfig) : getApps()[0];

export const auth = getAuth(app);
export const db = getFirestore(app);
```

El patrón `getApps().length === 0` evita inicializar Firebase múltiples veces en hot reload de Expo.

Colecciones Firestore

`users/{uid}`

Campo	Tipo	Descripción	
<code>email</code>	<code>string</code>	Email del usuario	
<code>displayName</code>	<code>string</code>	Nombre para mostrar	
<code>photoURL</code>	<code>`string`</code>	<code>null`</code>	URL de foto de perfil
<code>isActive</code>	<code>boolean</code>	Si el usuario puede acceder a la app	
<code>isBlocked</code>	<code>boolean</code>	Bloqueo manual por un administrador	
<code>roleIds</code>	<code>string[]</code>	IDs de roles asignados	
<code>permissionIds</code>	<code>string[]</code>	Permisos adicionales directos (fuera de roles)	
<code>lastLogin</code>	<code>`Timestamp`</code>	<code>null`</code>	Último inicio de sesión
<code>createdAt</code>	<code>Timestamp</code>	Fecha de creación	
<code>updatedAt</code>	<code>Timestamp</code>	Última modificación	
<code>updatedBy</code>	<code>string</code>	UID del usuario que modificó	

Reglas de acceso: Lectura por el propio usuario o quien tenga `manage:users` / `system:master` .
Borrado deshabilitado.

`roles/{roleId}`

Campo	Tipo	Descripción
<code>name</code>	<code>string</code>	Nombre del rol
<code>description</code>	<code>string</code>	Descripción del rol
<code>permissionIds</code>	<code>string[]</code>	IDs de permisos que otorga este rol
<code>order</code>	<code>number</code>	Orden de visualización (<code>-1</code> para System Master)
<code>isActive</code>	<code>boolean</code>	Si el rol está habilitado

Roles creados por el seed:

ID	Nombre	Descripción
<code>role_system_master</code>	System Master	Acceso total, gestiona colecciones base
<code>role_admin</code>	Administrador	Gestión completa de usuarios y pantallas
<code>role_gerente</code>	Gerente	Lectura de usuarios, roles y logs
<code>role_visualizador</code>	Visualizador	Solo lectura básica

`permissions/{permId}`

Campo	Tipo	Descripción
<code>key</code>	<code>string</code>	Identificador único (ej: <code>"manage:users"</code>)
<code>name</code>	<code>string</code>	Nombre legible
<code>description</code>	<code>string</code>	Qué permite hacer
<code>category</code>	<code>string</code>	Categoría (ej: <code>"users"</code> , <code>"screens"</code> , <code>"system"</code>)
<code>isActive</code>	<code>boolean</code>	Si el permiso está habilitado

Permisos creados por el seed:

Key	Descripción
<code>read:users</code>	Ver lista de usuarios

Key	Descripción
<code>manage:users</code>	Crear, editar y bloquear usuarios
<code>read:roles</code>	Ver roles y permisos
<code>manage:roles</code>	Crear y editar roles
<code>manage:screens</code>	Activar/desactivar/renombrar pantallas
<code>read:logs</code>	Ver logs de auditoría
<code>manage:config</code>	Modificar configuración de la app
<code>system:master</code>	Acceso total (solo para <code>role_system_master</code>)

`screens/{screenId}`

Campo	Tipo	Descripción	
<code>routePath</code>	<code>string</code>	Ruta Expo Router (ej: <code>"(tabs)/index"</code>)	
<code>displayName</code>	<code>string</code>	Nombre visible, editable desde admin	
<code>isActive</code>	<code>boolean</code>	Si la pantalla está habilitada	
<code>requiredPermissions</code>	<code>string[]</code>	Permission keys requeridas (lógica AND)	
<code>icon</code>	<code>`string`</code>	<code>null`</code>	Nombre de ícono
<code>order</code>	<code>number</code>	Orden en navegación	
<code>parentScreenId</code>	<code>`string`</code>	<code>null`</code>	Para pantallas anidadas

`logs/{logId}`

Auditoría inmutable de todas las acciones relevantes del sistema.

Campo	Tipo	Descripción	
<code>userId</code>	<code>string</code>	UID del actor	
<code>userEmail</code>	<code>string</code>	Email del actor	
<code>action</code>	<code>LogAction</code>	Tipo de acción	
<code>targetCollection</code>	<code>string</code>	Colección afectada	
<code>targetId</code>	<code>string</code>	Documento afectado	
<code>oldValue</code>	<code>`object`</code>	<code>null`</code>	Valor antes del cambio

Campo	Tipo	Descripción	
newValue	`object`	null`	Valor después del cambio
createdAt	Timestamp	Momento de la acción	

Actualización/Borrado: deshabilitado (immutable)**Acciones posibles (LogAction):**

login | logout | create | update | delete | read | block_user | unblock_user | assign_role | remove_role | assign_permission | remove_permission | enable_screen | disable_screen | change_screen_name

sessions/{sessionId}

Campo	Tipo	Descripción	
userId	string	UID del usuario	
loginAt	Timestamp	Inicio de sesión	
logoutAt	`Timestamp`	null`	Cierre de sesión (null si activa)
platform	string	Plataforma	
appVersion	string	Versión de la app	
isActive	boolean	Si la sesión está activa	

app_config/{key}

Campo	Tipo	Descripción
value	unknown	Valor de la configuración
description	string	Qué controla esta config
isPublic	boolean	Si es accesible sin permisos especiales

Creación/Actualización: solo system:master . Borrado: deshabilitado.

Script de seed (seed.js)

```
# Requiere service-account.json descargado de Firebase Console
GOOGLE_APPLICATION_CREDENTIALS=./service-account.json node seed.js
```

Crea: 8 permissions · 4 roles · 6+ screens · Configuración inicial de app_config

■ service-account.json contiene credenciales de administrador. **Nunca lo subas a git.**

Reglas de seguridad (firestore.rules)

Helper	Qué verifica
<code>isSignedIn()</code>	Usuario autenticado en Firebase Auth
<code>isAdmin()</code>	Campo <code>isAdmin == true</code> en el documento del usuario
<code>hasPermission(key)</code>	La key existe en <code>resolvedPermissionKeys</code> del usuario
<code>isAdminOrHasPermission(key)</code>	<code>isAdmin()</code> OR <code>hasPermission(key)</code>
<code>isSystemMaster()</code>	<code>hasPermission('system:master')</code>

5. Variables de entorno

La app usa el prefijo `EXPO_PUBLIC_` para exponer variables a la aplicación React Native.

Archivo `.env`

Crea este archivo en la raíz del proyecto. **No subir a git.**

```
EXPO_PUBLIC_FIREBASE_API_KEY=tu_api_key
EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN=tu_proyecto.firebaseio.com
EXPO_PUBLIC_FIREBASE_PROJECT_ID=tu_proyecto
EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET=tu_proyecto.firebaseio.com
EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=123456789
EXPO_PUBLIC_FIREBASE_APP_ID=1:123456789:web:abcdef
```

Cómo obtener estos valores:

- Ir a **Firebase Console**
- Seleccionar tu proyecto → **Configuración del proyecto**
- Bajar a "Tus apps" → seleccionar la app web
- Copiar los valores del objeto `firebaseConfig`

Dónde se usan

```
const firebaseConfig = {
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,
  storageBucket: process.env.EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET,
  messagingSenderId: process.env.EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,
};
```

`.gitignore` **recomendado**


```
.env
.env.local
.env.production
service-account.json
```

Variables del servidor (seed.js)

El script `seed.js` usa Firebase Admin SDK y requiere el archivo `service-account.json` :

```
GOOGLE_APPLICATION_CREDENTIALS=./service-account.json node seed.js
```

■■■ Este archivo contiene una clave privada RSA con acceso de administrador total al proyecto Firebase. Trátalo como una contraseña.

6. Sistema RBAC

Concepto general

```
Permission → tiene un "key" único (ej: "manage:users")
  ↑
Role → tiene un array de permissionIds
  ↑
AppUser → tiene roleIds[] + permissionIds[] directos
         permisos resueltos = union(permisos de roles + directos)
```

Los permisos se resuelven **en memoria** cuando el usuario inicia sesión, y se actualizan en tiempo real con `onSnapshot` .

Flujo de resolución de permisos

```

Firebase Auth onAuthStateChanged
  ■
  ▼
  onSnapshot(users/{uid})
    ■
    ■■■ isBlocked o !isActive? → logout automático
    ■
    ▼
    resolvePermissions(appUser)
      ■
      ■■■ getDocs(roles donde id in roleIds)
      ■■■ recolecta permissionIds de cada rol
      ■■■ agrega permissionIds directos del usuario
      ■■■ getDocs(permissions donde id in uniquePermIds)
      ■
      ▼
      setState({ roles, permissions, resolvedPermissionKeys: Set<string> })

```

Cómo usar permisos en componentes

Verificar un permiso (mostrar/ocultar UI):

```
import { usePermission } from "../hooks/useAuthHooks";

function MiComponente() {
  const puedeGestionarUsuarios = usePermission("manage:users");
  return puedeGestionarUsuarios ? <BotonEditar /> : null;
}
```

Proteger una pantalla completa:

```
import { useScreenGuard } from "../hooks/useAuthHooks";

export default function PantallaUsuarios() {
  const { checking } = useScreenGuard("pantallas/usuarios");
  if (checking) return <ActivityIndicator />;
  return <View>...</View>;
}
```

Acceder al usuario y sus datos completos:

```
import { useAuth } from "../context/AuthContext";

function MiPantalla() {
  const { appUser, roles, permissions, hasPermission, logout } = useAuth();
  return (
    <View>
      <Text>Hola, {appUser?.displayName}</Text>
      {hasPermission("read:logs") && <BotonVerLogs />}
    </View>
  );
}
```

Redirección global según autenticación:

```
import { useAuthRedirect } from "../hooks/useAuthHooks";

export default function RootLayout() {
  useAuthRedirect();
  return <Stack />;
}
```

Bloquear un usuario

```
import { setUserBlocked } from "../services/userService";

await setUserBlocked(uid, true, actorUid, actorEmail); // Bloquear
await setUserBlocked(uid, false, actorUid, actorEmail); // Desbloquear
```

El usuario bloqueado es desconectado automáticamente en el siguiente ciclo de `onSnapshot` .

Gestionar pantallas en tiempo real

```
import { renameScreen, setScreenActive } from "../services/screenService";

await renameScreen("screen_home", "Dashboard", actorUid, actorEmail);
await setScreenActive("screen_reportes", false, actorUid, actorEmail);
```

Pantallas del panel y sus permisos

Archivo	routePath	Permiso requerido
<code>pantallas/usuarios.tsx</code>	<code>pantallas/usuarios</code>	<code>manage:users</code>
<code>pantallas/roles.tsx</code>	<code>pantallas/roles</code>	<code>manage:roles</code>
<code>pantallas/pantallas.tsx</code>	<code>pantallas/pantallas</code>	<code>manage:screens</code>
<code>pantallas/logs.tsx</code>	<code>pantallas/logs</code>	<code>read:logs</code>
<code>pantallas/system-master.tsx</code>	<code>pantallas/system-master</code>	<code>system:master</code>

Lógica AND en pantallas

El campo `requiredPermissions` usa lógica **AND**: el usuario debe tener **todos** los permisos listados para acceder.

```
const hasAll = screen.requiredPermissions.every((key) =>
  resolvedPermissionKeys.has(key),
);
```

Notas de seguridad

- Las reglas de Firestore son la **última línea de defensa** — siempre activas independientemente de la lógica del cliente
- Para operaciones críticas, usar **Cloud Functions** con Admin SDK en lugar de escritura directa desde el cliente
- El log de auditoría (`Logs`) es **inmutable** desde el cliente

7. Componentes, Hooks y Tipos

`AuthContext.tsx` — Proveedor de autenticación

Ubicación: `context/AuthContext.tsx`

Provee el estado global de autenticación a toda la app. Escucha cambios en Firebase Auth y en el documento del usuario en Firestore en tiempo real.

Estado que expone (`AuthState`)

Campo	Tipo	Descripción	
<code>firebaseUser</code>	<code>`User`</code>	<code>null`</code>	Usuario de Firebase Auth
<code>appUser</code>	<code>`AppUser`</code>	<code>null`</code>	Documento del usuario en Firestore
<code>roles</code>	<code>Role[]</code>	Roles resueltos del usuario	
<code>permissions</code>	<code>Permission[]</code>	Permisos completos resueltos	
<code>resolvedPermissionKeys</code>	<code>Set<string></code>	Set de keys de permisos — verificación O(1)	
<code>isLoading</code>	<code>boolean</code>	Verdadero mientras carga la sesión inicial	
<code>isAuthenticated</code>	<code>boolean</code>	Verdadero si hay usuario autenticado y activo	

Métodos expuestos

Método	Firma	Descripción
<code>hasPermission</code>	<code>(key: string) => boolean</code>	Verifica si el usuario tiene un permiso
<code>logout</code>	<code>() => Promise<void></code>	Cierra sesión y registra en logs
<code>refreshAuth</code>	<code>() => void</code>	Fuerza recarga del usuario en Firebase Auth

Uso

```
// En app/_layout.tsx (raíz)
import { AuthProvider } from "../context/AuthContext";

export default function RootLayout() {
  return (
    <AuthProvider>
      <Stack />
    </AuthProvider>
  );
}

// En cualquier componente hijo
import { useAuth } from "../context/AuthContext";
const { appUser, hasPermission, logout } = useAuth();
```

Helper interno: chunkArray

Divide arrays en chunks de tamaño `n` para respetar el límite de 10 elementos por query en Firestore.

```
chunkArray(["a","b","c","d"], 2) // → [["a","b"], ["c","d"]]
```

useAuthHooks.ts — Hooks de autenticación

Ubicación: `hooks/useAuthHooks.ts`

`usePermission(permissionKey: string): boolean`

```
const puedeEditar = usePermission("manage:users");
// → true | false
```

`useScreenGuard(routePath: string)`

Protege una pantalla completa verificando en Firestore si la pantalla está activa y si el usuario tiene los permisos necesarios.

- Si la pantalla no existe en Firestore → acceso libre (modo desarrollo)
- Si `isActive: false` → redirige a `/(tabs)`
- Si el usuario no tiene todos los `requiredPermissions` → redirige a `/(tabs)`

```
export default function PantallaAdmin() {
  const { checking } = useScreenGuard("pantallas/usuarios");
  if (checking) return <ActivityIndicator />;
  return <View>{/* contenido */</View>;
}
```

`useAuthRedirect()`

Maneja las redirecciones globales basadas en el estado de autenticación.

- Si no está autenticado y está en `(tabs)` → redirige a `/bienvenida/bienvenida`
- Si está autenticado y está en `bienvenida` o `login` → redirige a `/(tabs)`

lib/types.ts — Tipos TypeScript

Permission

```
interface Permission {
  id: string;
  name: string;
  description: string;
  key: string;          // ej: "read:users", "manage:screens"
  isActive: boolean;
  category: string;
  createdAt: Timestamp;
  updatedAt: Timestamp;
  createdBy: string;
  updatedBy: string;
}
```

Role

```
interface Role {
  id: string;
  name: string;
  description: string;
  isActive: boolean;
  permissionIds: string[];
  order: number;        // -1 para System Master (aparece primero)
  createdAt: Timestamp;
  updatedAt: Timestamp;
  createdBy: string;
  updatedBy: string;
}
```

AppUser

```
interface AppUser {
  uid: string;
  email: string;
  displayName: string;
  photoURL: string | null;
  isActive: boolean;
  isBlocked: boolean;
  roleIds: string[];
  permissionIds: string[];
  lastLogin: Timestamp | null;
  createdAt: Timestamp;
  updatedAt: Timestamp;
  updatedBy: string;
}
```

Screen

```
interface Screen {
  id: string;
  routePath: string;
  displayName: string;
  isActive: boolean;
  requiredPermissions: string[];
  icon: string | null;
  order: number;
  parentScreenId: string | null;
  createdAt: Timestamp;
  updatedAt: Timestamp;
  updatedBy: string;
}
```

Log

```
interface Log {
  id: string;
  userId: string;
  userEmail: string;
  action: LogAction;
  targetCollection: string;
  targetId: string;
  oldValue: Record<string, unknown> | null;
  newValue: Record<string, unknown> | null;
  createdAt: Timestamp;
  ipAddress: string | null;
  platform: string;
}
```

Session

```
interface Session {
  id: string;
  userId: string;
  loginAt: Timestamp;
  logoutAt: Timestamp | null;
  platform: string;
  appVersion: string;
  ipAddress: string | null;
  isActive: boolean;
  deviceInfo: string | null;
}
```

AppConfig

```
interface AppConfig {
  id: string;
  value: unknown;
  description: string;
  updatedAt: Timestamp;
  updatedBy: string;
  isPublic: boolean;
}
```

LogAction (union type)

```
type LogAction =
  | "login" | "logout"
  | "create" | "update" | "delete" | "read"
  | "block_user" | "unblock_user"
  | "assign_role" | "remove_role"
  | "assign_permission" | "remove_permission"
  | "enable_screen" | "disable_screen" | "change_screen_name";
```

8. Pantallas

Estructura de navegación

```
app/
  ■■■ _layout.tsx          ← Root layout (Stack) + AuthProvider
  ■■■ bienvenida/
  ■   ■■■ bienvenida.tsx   ← Pantalla de bienvenida (pública)
  ■■■ login/
  ■   ■■■ login.tsx        ← Inicio de sesión
  ■   ■■■ register.tsx     ← Registro de cuenta
  ■   ■■■ forgot.tsx       ← Recuperar contraseña
  ■■■ (tabs)/
  ■   ■■■ _layout.tsx      ← Bottom tabs (nombres y visibilidad desde Firestore)
  ■   ■■■ index.tsx        ← Dashboard / Home
  ■   ■■■ profile.tsx      ← Perfil del usuario
  ■■■ pantallas/
  ■   ■■■ _layout.tsx      ← Stack con header para el panel admin
  ■   ■■■ usuarios.tsx     ← Gestión de usuarios [manage:users]
  ■   ■■■ roles.tsx        ← Gestión de roles [manage:roles]
  ■   ■■■ gestionar-pantallas.tsx ← Gestión de pantallas [manage:screens]
  ■   ■■■ logs.tsx         ← Auditoría [read:logs]
  ■   ■■■ system-master.tsx ← Panel System Master [system:master]
  ■   ■■■ pantalla1.tsx    ← Pantalla de ejemplo
```

Flujo de navegación general


```

App inicia → _layout.tsx (RootLayout)
■
  ■■■ isLoading → ActivityIndicator (spinner)
  ■■■ No autenticado → /bienvenida/bienvenida
  ■    ■■■ → /login/login → /login/register | /login/forgot
  ■■■ Autenticado → /(tabs)
      ■■■ index (Dashboard)
      ■■■ profile (Perfil)
          ■■■ pantallas/ (Panel admin, según permisos)

```

Pantallas públicas

bienvenida/bienvenida.tsx

Pantalla de entrada para usuarios no autenticados. Contiene título principal, botón "Entrar" → `/login/login`, y sección "Saber más" con modal Markdown.

login/login.tsx

Campos: email, contraseña. Llama a `loginUser(email, password)` de `authService`. Redirige automáticamente a `/(tabs)` si es exitoso.

Manejo de errores (`friendlyError`):

Código Firebase	Mensaje mostrado
<code>invalid-credential</code> / <code>wrong-password</code>	Correo o contraseña incorrectos
<code>user-not-found</code>	No existe cuenta con ese correo
<code>too-many-requests</code>	Demasiados intentos, espera
<code>network-request-failed</code>	Sin conexión a internet

login/register.tsx

Campos: nombre completo, email, contraseña, selector de rol. El rol se incluye en la creación inicial del usuario (no se llama `updateUserRoles` después para evitar errores de permisos).

login/forgot.tsx

Campo: email. Llama a `resetPassword(email)` de `authService`. Firebase envía un correo con enlace de restablecimiento.

Pantallas principales (`tabs`)

(`tabs`)/index.tsx — Dashboard / Home

Muestra saludo personalizado y tarjetas de las pantallas de administración accesibles para el usuario (filtradas por permisos en tiempo real con `onSnapshot`).

Iconos por pantalla:

routePath	Ícono
<code>pantallas/usuarios</code>	■

routePath	Ícono
pantallas/roles	■ ■
pantallas/pantallas	■ ■
pantallas/logs	■
cualquier otro	■

(tabs)/profile.tsx — Perfil

Secciones: avatar + nombre con badge de estado, roles asignados, permisos efectivos, información de cuenta, botón de cerrar sesión.

Generación de iniciales:

```
displayName.split(" ").map(n => n[0]).slice(0, 2).join("").toUpperCase()
// "Jorge Alejandro" → "JA"
```

Panel de administración pantallas/

Todas usan `useScreenGuard(routePath)` al inicio y muestran spinner mientras verifica permisos.

pantallas/usuarios.tsx — Gestión de usuarios

Permiso requerido: `manage:users`

Acción	Método
Ver usuarios	<code>getAllUsers()</code>
Bloquear/desbloquear	<code>setUserBlocked(uid, bool, actorUid, actorEmail)</code>
Asignar roles	<code>updateUserRoles(uid, roleIds[], actorUid, actorEmail)</code>

pantallas/roles.tsx — Gestión de roles

Permiso requerido: `manage:roles`

Acción	Método
Crear rol	<code>createRole(data, actorUid, actorEmail)</code>
Editar	<code>updateRole(id, data, actorUid, actorEmail)</code>
Activar/desactivar	<code>updateRole(id, { isActive }, actorUid, actorEmail)</code>

pantallas/gestionar-pantallas.tsx — Gestión de pantallas

Permiso requerido: `manage:screens`

Acción	Método
Activar/desactivar	<code>setScreenActive(id, bool, actorUid, actorEmail)</code>
Renombrar	<code>renameScreen(id, name, actorUid, actorEmail)</code>
Editar permisos	<code>updateScreenPermissions(id, permKeys[], actorUid, actorEmail)</code>

`pantallas/logs.tsx` — Auditoría

Permiso requerido: `read:logs` . Los logs son inmutables — no se pueden editar ni borrar desde el cliente.

`pantallas/system-master.tsx` — Panel System Master

Permiso requerido: `system:master` . Panel exclusivo para `role_system_master` . Permite gestionar las colecciones base del sistema.

Cómo agregar una nueva pantalla al panel admin

- Crear el archivo en `app/pantallas/nueva-pantalla.tsx`
- Agregar `useScreenGuard("pantallas/nueva-pantalla")` al inicio:

```
import { useScreenGuard } from "@/hooks/useAuthHooks";

export default function MiPantalla() {
  const { checking } = useScreenGuard("pantallas/mi-pantalla");
  if (checking) return <ActivityIndicator />;

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Mi Pantalla</Text>
    </View>
  );
}
```

- Registrar la pantalla en Firestore:

```
{
  "routePath": "pantallas/nueva-pantalla",
  "displayName": "Mi Nueva Pantalla",
  "isActive": true,
  "requiredPermissions": ["manage:users"],
  "order": 10
}
```

- Opcionalmente agregar su ícono emoji en `SCREEN_ICONS` de `index.tsx` .

La pantalla aparecerá automáticamente en el Dashboard de los usuarios con los permisos necesarios.

Componente compartido: Card

Utilizado en `login.tsx` , `register.tsx` y `forgot.tsx` .

Prop	Tipo	Descripción
<code>title</code>	<code>string</code>	Título del formulario
<code>subtitle</code>	<code>string</code>	Subtítulo descriptivo
<code>fields</code>	<code>Field[]</code>	Campos de texto
<code>selects</code>	<code>Select[]</code>	Selectores desplegables
<code>primaryButton</code>	<code>{ label, onPress }</code>	Botón principal de acción
<code>secondaryButton</code>	<code>{ label, onPress }</code>	Botón secundario (link)
<code>theme</code>	<code>{ primaryBackground, borderRadius }</code>	Personalización visual

Documentación generada para el proyecto `h_app` · Expo SDK 54 · Firebase v12 · TypeScript strict